Unified Production-Grade ML Churn Repository (No Redundancy)

Perspective: Production Ready Code
Goal: Build a single, perfect repository that is safe for training, evaluation, inference, CI/CD, and long-term production use.

This repository:

- Eliminates redundancy completely
- Saves one pipeline artifact (preprocessing + model)
- Enforces schema + data quality
- Detects data drift
- Fails fast instead of silently corrupting models

1️⃣ Final Folder Structure (Industry Standard)

```
ml_churn_project/
│
├── data/
│   ├── raw/                    # Immutable source data
│   ├── reference/              # Baseline data for drift detection
│   └── current/                # Incoming production data
│
├── models/
│   └── churn_pipeline_v1.pkl   # Preprocessing + model (single artifact)
│
├── reports/
│   └── drift_report.json       # Drift metrics
│
├── src/
│   ├── config.py
│   ├── schema.py
│   ├── data_validation.py
│   ├── drift_detection.py
│   ├── preprocessing.py
│   ├── train.py
│   ├── evaluate.py
│   └── inference.py
│
├── main.py
└── requirements.txt
```

> Every file exists for exactly one reason. No file duplicates another file's responsibility.

2️⃣ config.py — Single Source of Truth

```
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent

RAW_DATA = BASE_DIR / "data/raw/telco_churn.csv"
REFERENCE_DATA = BASE_DIR / "data/reference/reference.csv"
CURRENT_DATA = BASE_DIR / "data/current/current.csv"

MODEL_PATH = BASE_DIR / "models/churn_pipeline_v1.pkl"
DRIFT_REPORT_PATH = BASE_DIR / "reports/drift_report.json"

TARGET_COL = "Churn"
RANDOM_STATE = 42
```

Explanation (line-by-line):

- `Pathlib` avoids OS-specific path bugs
- All paths live in one file → zero hard-coding
- Changing environments requires no code edits

3️⃣ schema.py — Explicit Data Contract

```
SCHEMA = {
    "gender": "object",
    "SeniorCitizen": "int",
    "Partner": "object",
    "Dependents": "object",
    "tenure": "int",
    "PhoneService": "object",
    "InternetService": "object",
    "MonthlyCharges": "float",
    "TotalCharges": "float",
    "Churn": "object"
}
```

Why this matters:

- ML systems break silently without schemas
- This prevents upstream teams from breaking your model
- This is a hard contract, not documentation

4️⃣ data_validation.py — Quality + Schema Validation

```
import pandas as pd
from src.schema import SCHEMA
from src.config import TARGET_COL

def validate(df: pd.DataFrame):
    missing_cols = set(SCHEMA) - set(df.columns)
    if missing_cols:
        raise ValueError(f"Missing columns: {missing_cols}")

    for col, dtype in SCHEMA.items():
        if dtype == "int" and not pd.api.types.is_integer_dtype(df[col]):
            raise TypeError(f"{col} must be integer")

    assert df.isnull().mean().max() < 0.4, "Too many missing values"
    assert df.duplicated().sum() == 0, "Duplicate rows found"
    assert TARGET_COL in df.columns, "Target column missing"

    return df
```

Why this exists:

- Schema validation ≠ data quality validation
- Both must pass before training
- Fail fast & loud is a production rule

5️⃣ preprocessing.py — Leakage-Safe Feature Engineering

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer

NUM_COLS = ["tenure", "MonthlyCharges", "TotalCharges"]
CAT_COLS = [
    "gender", "Partner", "Dependents",
    "PhoneService", "InternetService"
]

numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("num", numeric_pipeline, NUM_COLS),
    ("cat", categorical_pipeline, CAT_COLS)
])
```

Key production rules enforced:

- No leakage (scaling inside pipeline)
- No crashes on unseen categories
- Numeric ≠ categorical processing

6️⃣ train.py — Single Persisted Pipeline

```
import joblib
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline

from src.config import RAW_DATA, MODEL_PATH, TARGET_COL, RANDOM_STATE
from src.preprocessing import preprocessor
from src.data_validation import validate

def train():
    df = pd.read_csv(RAW_DATA)
    df[TARGET_COL] = df[TARGET_COL].map({"Yes": 1, "No": 0})

    validate(df)

    X = df.drop(TARGET_COL, axis=1)
    y = df[TARGET_COL]

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, stratify=y, random_state=RANDOM_STATE
    )

    model = RandomForestClassifier(
        n_estimators=200,
        random_state=RANDOM_STATE,
        n_jobs=-1
    )

    pipeline = Pipeline([
        ("preprocessing", preprocessor),
        ("model", model)
    ])

    pipeline.fit(X_train, y_train)
    joblib.dump(pipeline, MODEL_PATH)

    return pipeline, X_test, y_test
```

> **Rule:** You never save a model alone. You save the pipeline — always.

7️⃣ drift_detection.py — Statistical Data Drift

```
import json
import pandas as pd
from scipy.stats import ks_2samp

from src.config import REFERENCE_DATA, CURRENT_DATA, DRIFT_REPORT_PATH
from src.preprocessing import NUM_COLS

def detect_drift():
    ref = pd.read_csv(REFERENCE_DATA)
    cur = pd.read_csv(CURRENT_DATA)

    drift_results = {}

    for col in NUM_COLS:
        stat, p_value = ks_2samp(ref[col], cur[col])
        drift_results[col] = {
            "p_value": float(p_value),
            "drift_detected": p_value < 0.05
        }

    with open(DRIFT_REPORT_PATH, "w") as f:
        json.dump(drift_results, f, indent=2)

    return drift_results
```

Why KS-test:

- Non-parametric
- Detects distribution shift
- Production-proven

8️⃣ evaluate.py — Honest Metrics

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score

def evaluate(model, X_test, y_test):
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1]

    return {
        "accuracy": accuracy_score(y_test, y_pred),
        "precision": precision_score(y_test, y_pred),
        "recall": recall_score(y_test, y_pred),
        "f1": f1_score(y_test, y_pred),
        "roc_auc": roc_auc_score(y_test, y_prob)
    }
```

9️⃣ inference.py — Production-Safe Prediction

```
import joblib
import pandas as pd
from src.config import MODEL_PATH

pipeline = joblib.load(MODEL_PATH)

def predict(input_dict: dict):
    df = pd.DataFrame([input_dict])
    return pipeline.predict(df)[0]
```

🔟 main.py — Orchestration

```
from src.train import train
from src.evaluate import evaluate
from src.drift_detection import detect_drift

if __name__ == "__main__":
    model, X_test, y_test = train()
    metrics = evaluate(model, X_test, y_test)
    drift = detect_drift()

    print("Metrics:", metrics)
    print("Drift:", drift)
```

🧠 Final Truth

> This is not tutorial ML. This is **production ML architecture**.